

4

Grab That Joystick – Input and Event Management

Arguably, the most important aspect of any game ever made is actually being able to play it. Regardless of the purpose of input, ranging from simply hitting keys to navigating through menus to controlling when your character jumps and which direction he or she walks to, the lack of an application presenting a way for you to interact with it might as well leave you with a very fancy screensaver. We have very briefly looked at the primitive way of grabbing and using the keyboard input, however our motivation for this chapter is quite different than simply being content with a large nest of if/else statements that handle every single key being pressed. Instead, we want to look at a more robust way of handling not just the keyboard, but also the mouse and any events that happen between the frames, along with adding potential for processing input of additional peripherals, such as joysticks. With that in mind, let's take a look at what we will be covering in this chapter:

- Basic means of checking the states of keyboard and mouse buttons
- Understanding and processing different types of events
- Understanding and utilizing callbacks
- Designing and implementing an event manager

Let's not sit still like the character of your game without input and get on coding!

Retrieving peripheral input

A few of the previous chapters have already touched on this subject of retrieving peripheral output a little bit, and, ironically enough, the entire scope of the class was covered. Just to recap, `sf::Keyboard` is a class that provides a single static method `isKeyPressed(sf::Keyboard::Key)` to determine the real-time state of a certain keyboard key, which gets passed in as an argument to the method, represented by the `sf::Keyboard::Key` enumeration table. Because this method is static, `sf::Keyboard` doesn't need to be instantiated and can be used as follows:

```
if(sf::Keyboard::isKeyPressed(sf::Keyboard::W)){
    // Do something if the W key is pressed.
}
```

This is the way we checked for input in the previous chapters, however, it does lend itself to quite a bit of a mess of `if/else` statements if we want to check for more keystrokes.

Checking for mouse input

Predictably enough, SFML also provides a class similar to `sf::Keyboard` with the same idea of obtaining real-time status of a mouse: `sf::Mouse`. Much like its partner in crime, the keyboard, it provides a way to check for the mouse buttons being pressed, as shown next:

```
if(sf::Mouse::isButtonPressed(sf::Mouse::Left)){
    // Do something if the left mouse button is pressed.
}
```

The `sf::Mouse` class provides its own enumeration of possible buttons on any given mice, of which we have a grand total of five:

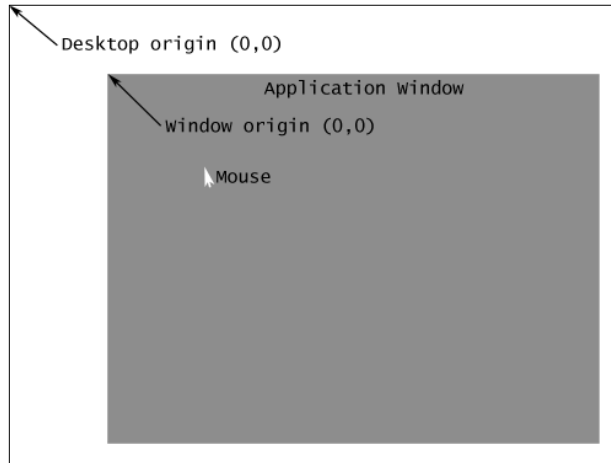
<code>sf::Mouse::Left</code>	The left mouse button
<code>sf::Mouse::Right</code>	The right mouse button
<code>sf::Mouse::Middle</code>	Mouse wheel being clicked
<code>sf::Mouse::XButton1</code>	First extra mouse button
<code>sf::Mouse::XButton2</code>	Second extra mouse button

In addition to that, the `sf::Mouse` class provides a way to get and set the current mouse position:

```
// Getting the mouse position.
sf::Vector2i mousePos = sf::Mouse::getPosition(); // 1
sf::Vector2i mousePos = sf::Mouse::getPosition(m_window); // 2
```

```
// Setting the mouse position.  
sf::Mouse::setPosition(sf::Vector2i(0,0)); // 3  
sf::Mouse::setPosition(sf::Vector2i(0,0),m_window); // 4
```

Both these methods have an overloaded version that takes in a reference to a window in order to determine whether to look at the mouse coordinates relative to the window or relative to the desktop. Consider the following illustration:



If the reference to a window isn't provided, like on line #1 of the previous example, the mouse position that gets returned is the distance from the desktop origin to the point the mouse is at. If, however, the reference to a window is provided, the position is simply the distance between the window origin and the mouse location. In other words, the mouse position in the example #2 is relative to the window. The same logic is true for lines #3 and #4, except the position of the mouse gets set to the provided int vector argument.

Plug in your controller

Yes, as the title states, SFML supports input not only from your keyboard and mouse, but also from additional peripherals that you may have hooked up to your computer. By utilizing the class `sf::Joystick`, which only contains static methods, just like the previous two classes, it is possible to check if a controller is connected, check for its button states, and even determine the positions along certain axes, if the controller supports that.

SFML supports up to eight different controllers being connected at the same time, which are identified by a numerical index in the range of [0;7]. Because of that, every method that `sf::Joystick` provides has to have at least one argument, which is the controller ID. First, let's take a look at a way to determine if a controller is connected:

```
if (sf::Joystick::isConnected(0))
{
    // We have a controller with an id 0.
}
```

If we have a controller with an ID of 0, we can check how many buttons it actually supports, as follows:

```
unsigned int n_buttons = sf::Joystick::getButtonCount(0);
```

Because there is no other way to abstractly define the buttons for every controller on the planet, they're simply referred to by numeric indices between 0 and 31. Checking for a button push can be done by calling the `isButtonPressed()` method, as shown next:

```
if (sf::Joystick::isButtonPressed(0,1)) {
    // Button 1 on controller 0 is pressed.
}
```

In order to check if a controller supports a specific axis, we can use the `hasAxis()` method:

```
if (sf::Joystick::hasAxis(0,sf::Joystick::X)) {
    // Controller 0 supports movement on X axis.
}
```

The `sf::Joystick::Axis` enumeration encapsulates all the possible axes that a controller could support, so one can check for that as shown in the preceding code. Assuming the controller supports it, obtaining its current position along an axis can be done as follows:

```
float p_x = sf::Joystick::getAxisPosition(0, sf::Joystick::X);
float p_y = sf::Joystick::getAxisPosition(0, sf::Joystick::Y);
// Do something with p_x and p_y.
```

The preceding methods will return the current position of the X and Y axes on the controller 0.



Because the `sf::Joystick` states are updated when checking for events, it might present some problems when using any of these methods before the events have had a chance to be polled. If that's the case, it is best to manually call the `sf::Joystick::Update()` method in order to make sure you have the latest state of your peripherals.

Understanding the `sf::Event`

Once again, `sf::Event` is something we briefly touched on, however, it's imperative to expand on it and understand it better before proceeding, if we want to build a system that can seamlessly handle all types of events without any hiccups. First, let's reiterate what an event is. `sf::Event` is a union, which in C++ terms means that it's a special class which can hold only one of its non-static data members at a time, of which it has several, such as `KeyEvent`, which holds the information about a keyboard event, `SizeEvent`, which holds information about the size of our window that got resized, and many others. Because of this nature of `sf::Event`, it can be a trap for newcomers if they handle the event in a wrong way, such as accessing the `KeyEvent` struct inside `sf::Event`, when that is not the active data member. Since all the members of a union share the *same memory space*, this results in undefined behavior and will crash your application, unless you know what you're doing.

Let's take a look at the most basic way of processing events:

```
sf::Event event;

while(m_window.pollEvent(event)) {
    switch(event.type) {
        case sf::Event::Closed:
            m_window.close();
            break;

        case sf::Event::KeyPressed:
            if(event.key.code == sf::Keyboard::W) {
                // Do something when W key gets pressed once.
            }
            break;
    }
}
```

Nothing we haven't seen before, although it's important we fill in the blanks of what exactly is going on. First, the `sf::Event` instance named `event` gets filled out by the `pollEvent()` method. Based on its type, it will choose one of the structures in the union to be the active one to carry the data relevant to the event. Afterwards, we can check for the type of the event, which is defined by the `sf::Event::Type` enumeration table and make sure we're using the correct data member to obtain the information we need. As mentioned before, trying to access `event.key.code` if the event type is `sf::Event::Closed`, for example, would result in an undefined behavior.



Remember, using the `sf::Event::KeyPressed` event for something like real-time character movement is a bad idea. This event gets dispatched only once before a small delay is applied and then it gets dispatched again. Think of a document editor here. When you press down on a key and hold it, at first it only shows a single character before it writes more. This is exactly the same way this event works. Using it for any action that needs to be continuous as long as the key is being held down is not even close to optimal and should be replaced with `sf::Keyboard::isKeyPressed()` in order to check the actual state of the key. The same idea applies to the mouse and controller input. Using these events is ideal for things that only need to happen once per keystroke, but not much else.

While this approach is manageable in cases of small projects, pretty much the same as the input example was before, it can get out of hand quickly on a larger scale. Let's face it, handling all the events, keystrokes, and states of every input device the way we did in the previous project is a nightmare. Still not convinced? Imagine having an application where you want to check for multiple keys being pressed at the same time and call some function when they are. Not too bad? Well, let's include events in that scenario. You want to check for two keys being pressed and a certain event taking place at the same time, in order to call a function. That adds another layer of complexity, but nothing you can't handle, right? Throwing in some Boolean flags in there to keep track of the event states or maybe the keystrokes shouldn't be too hard.

Some time passes and the application now needs support for loading key combinations from a file in order to make your approach more dynamic and customizable. You have a mess on your hands. You can build it, but it's going to be so awkward to add new functionality or expand that mountain of nonsense that you are likely to just throw your arms in the air and give up. Why put yourself through all of that when with just some effort and white-boarding you can come up with an automated approach that will need no flags, is flexible, can load any combination of keys and events from a file, and still keep your code just as neat and clean as it was before, if not more so? Let's solve this problem intelligently by working on a system that will handle all of these headaches for us.

Introducing the event manager

Figuring out what we want from our application is the first and the most crucial part of the design process. Sometimes it's difficult to cover all your bases, but forgetting about a feature that might alter the way all the code is structured and trying to implement it later can wreak some serious havoc on all the work you put into your software. Having said that, let's make a list of what features we want our event manager to have:

- The ability to couple any mix of keys, buttons, or events (from now on referred to as bindings) with desired functionality identified by a string
- Binding of the said functionalities to methods that get called if all the conditions (such as a key being pressed, the left mouse button being clicked, or the window losing focus, for example) for the binding are satisfied
- A way through which the event manager can deal with actual SFML events being polled
- Loading the bindings from a configuration file

We have our specifications, now let's start designing! We'll be using the `EventManager.h` file to include all the little bits and pieces that make this possible on top of having the definition of the class. The first thing that we need to define is all the types of events we'll be dealing with. This can be extended later on, but as this will more than suit our purposes for now, we don't need to worry about that just yet. Let's write the enumeration table:

```
enum class EventType{
    KeyDown = sf::Event::KeyPressed,
    KeyUp = sf::Event::KeyReleased,
    MButtonDown = sf::Event::MouseButtonPressed,
    MButtonUp = sf::Event::MouseButtonReleased,
    MouseWheel = sf::Event::MouseWheelMoved,
    WindowResized = sf::Event::Resized,
    GainedFocus = sf::Event::GainedFocus,
    LostFocus = sf::Event::LostFocus,
    MouseEntered = sf::Event::MouseEntered,
    MouseLeft = sf::Event::MouseLeft,
    Closed = sf::Event::Closed,
    TextEntered = sf::Event::TextEntered,
    Keyboard = sf::Event::Count + 1, Mouse, Joystick
};
```

The majority of these are actual events; however, note the last row before the enumeration is terminated. We're setting up our own event, called `Keyboard` to the value of `sf::Event::Count + 1`. Because all the enumerations are essentially keywords pointing to integer values, the last row prevents any kind of identifier clashing and makes sure that anything added past this point is higher than the absolute maximum `sf::Event::EventType` enumeration value. As long as anything added before the last row is a valid event type, there should be no clashes.



The `sf::Event` enumeration values can be different, depending on which version of SFML you are using!

Next, let's make it possible to store these groups of events for each binding. We know that in order to bind to a key, we need both the event type and the code for the key that we're interested in. Some events we'll be working with only need to have a type stored, in which cases we can simply store an integer value of 0 with the type. Knowing that, let's define a new structure that will help us store this information:

```
struct EventInfo{
    EventInfo(){ m_code = 0; }
    EventInfo(int l_event){ m_code = l_event; }
    union{
        int m_code;
    };
};
```

In order to leave room for expansions, we're already using a **union** to store the event code. Next, we can set up the data type that we're going to be using to hold the event information:

```
using Events = std::vector<std::pair<EventType, EventInfo>>;
```

Since we're going to need to share the event information with the code that uses this class, now is as good a time as any to set up a data type that will help us do that:

```
struct EventDetails{
    EventDetails(const std::string& l_bindName)
        : m_name(l_bindName)
    {
        Clear();
    }
    std::string m_name;

    sf::Vector2i m_size;
    sf::Uint32 m_textEntered;
```

```

    sf::Vector2i m_mouse;
    int m_mouseWheelDelta;
    int m_keyCode; // Single key code.

    void Clear(){
        m_size = sf::Vector2i(0, 0);
        m_textEntered = 0;
        m_mouse = sf::Vector2i(0, 0);
        m_mouseWheelDelta = 0;
        m_keyCode = -1;
    }
};

```

Now it's time to design the binding structure, which is going to hold all the event information. Seems quite simple, so let's implement it:

```

struct Binding{
    Binding(const std::string& l_name)
        : m_name(l_name), m_details(l_name), c(0){}
    void BindEvent(EventType l_type,
        EventInfo l_info = EventInfo())
    {
        m_events.emplace_back(l_type, l_info);
    }

    Events m_events;
    std::string m_name;
    int c; // Count of events that are "happening".

    EventDetails m_details;
};

```

The constructor takes the name of the action we want to bind the events to and uses the initializer list to set up the class data members. We also have a `BindEvent()` method, which simply takes in an event type and an event information structure in order to add it to the event vector. One additional data member that we haven't mentioned before is the integer with the name `c`. As the comment suggests, this keeps track of how many events are actually taking place, which will be useful later on in order to determine if all the keys and events in the binding are "on". Lastly, this is the structure where the event detail data member that gets shared around resides.

These bindings will also have to be stored somehow, so let's define the data type for the container that will take care of it:

```
using Bindings = std::unordered_map<std::string, Binding*>;
```

Using `std::unordered_map` for our bindings guarantees that there will only be one binding per action, since it's an associative container and the action name string is the key for that container.

We're doing great so far, however, without a way to actually tie these actions to valid methods that will get called, this system is fairly useless. Let's talk about how we could implement this. In the world of computer science, every now and then you've probably heard the term "callback" being thrown around. In simplest terms, a callback is some chunk of code that gets passed as an argument to another piece of code, which *will* execute it at a convenient time. In the case of our event manager, the convenient time is whenever all the events that are bound to a specific action are happening, and the callback is a method that represents the action being performed. Let's say, we want the character to jump when the spacebar is hit. We would create a binding with a name "Jump", which is our action name, and add a single event of type `KeyDown` and code `sf::Keyboard::Space` to it. For argument sake, let's say the character has a method called `Jump()`. That's our callback. We want to bind that method to the name "Jump" and have the event manager call the character's `Jump()` method when the space key is pressed. That, in a nutshell, is how we're going to be handling input with this new system.

By now, your C++ background is probably driving you towards the term "function pointers". While that's not necessarily a bad option, it can get a little messy if you're quite new to the scene. The main problem with that approach is the scenario of adding a method of a class as a callback. Pointers to the class members aren't exactly the same as regular functions, unless it's a static method. Following is a basic definition of a member function pointer:

```
void(SomeClass::*_callback)();
```

Already this shows a few major limitations. For one, we can only have pointers to methods of the class "SomeClass". Secondly, without having an instance to the class that has the method we're pointing to, it's quite useless. A thought has probably popped into your mind of just storing the instance together with the function pointer in some callback structure. Let's take a look:

```
struct Callback{
    std::string m_name;
    SomeClass* CallbackInstance; // Pointer to instance.
    void(SomeClass::*_callback)();
```

```
void Call() {  
    CallbackInstance->*_callback();  
}  
};
```

That's a little better. At least we can call the method now, although we're still limited to only one class. We could just wrap every other class method call in the methods of the "SomeClass" class, but that's tedious and more importantly, it's a bad practice. Maybe now you're thinking that some template magic might solve this problem. While it is possible, you have to also take into account the compatibility and the mess that it might create. Consider the most minimum amount of effort this could possibly take:

```
template<class T>  
struct Callback{  
    ...  
    T* CallbackInstance; // Pointer to instance.  
    void(T::*_callback)();  
    ...  
};
```

This by itself doesn't solve anything, but instead it only brings more problems. For one, you now have to define that template in your event manager class, which is problematic because we need a container for all these callbacks and that means having to template the entire event manager class, which locks it down to one class type. We're right back to where we started. Using typedef would be a clever idea, except that it's not supported in most of the Visual Studio compilers in this form:

```
template <class T>  
using Function = void (T::*)();
```

There are some hackish workarounds for non C++11 compilers, like wrapping typedef in struct after defining the template. However, that doesn't solve our problem either. There have been instances of the Visual Studio 2010 compiler even crashing when using "templated" member function pointer type definitions. This is quite a mess, and at this point you're probably thinking about simply going back to regular function pointers and wrapping every single member function call in a different function. Fear not, C++11 introduces a much better approach than that.

Standard function wrapper

The C++ utilities library provides us with just what we need in order to solve this pickle elegantly: `std::function` and `std::bind`. The `std::function` type is a general purpose polymorphic function wrapper. Amongst many other things it supports, it can store the member function pointers and call them. Let's take a look at a minimal example of using it:

```
#include <functional> // Defines std::function & std::bind.
...
std::function<void(void)> foo = std::bind(&Bar::method1, this);
```

In this case, we're instantiating a function wrapper called "foo", which holds a function with the signature `void(void)`. On the right side of the equals sign, we use `std::bind` to bind the member function "method1" of the class "Bar" to the `foo` object. The second argument, because this is a member function pointer, is the instance of the class that is having its method registered as a callback. In this case, it has to be an instance of the `Bar` class, so let's imagine this line of code is written somewhere in the implementation of it and just pass in "this". Now our `foo` object is bound to the method `method1` of class `Bar`. Because `std::function` overloads the parenthesis operator, calling it is as easy as this:

```
foo(); // Equivalent to barInstance->method1();
```

Now we can finally define the type of the callback container:

```
using Callbacks = std::unordered_map<std::string,
std::function<void(EventDetails*)>>;
```

Once again, using `std::unordered_map` ensures that there's only one callback per action. This can be changed later if needed.

Building the event manager

At this time, we have everything we need to actually write the header of our event manager class. Given all the design decisions we made previously, it should come out looking something like the following:

```
class EventManager{
public:
    EventManager();
    ~EventManager();

    bool AddBinding(Binding *l_binding);
    bool RemoveBinding(std::string l_name);
```

```

void SetFocus(const bool& l_focus);

// Needs to be defined in the header!
template<class T>
bool AddCallback(const std::string& l_name,
    void(T::*l_func)(EventDetails*), T* l_instance)
{
    auto temp = std::bind(l_func, l_instance,
        std::placeholders::_1);
    return m_callbacks.emplace(l_name, temp).second;
}

void RemoveCallback(const std::string& l_name) {
    m_callbacks.erase(l_name);
}

void HandleEvent(sf::Event& l_event);
void Update();

sf::Vector2i GetMousePos(sf::RenderWindow* l_wind = nullptr) {
    return (l_wind ? sf::Mouse::getPosition(*l_wind)
        : sf::Mouse::getPosition());
}

private:
    void LoadBindings();

    Bindings m_bindings;
    Callbacks m_callbacks;
    bool m_hasFocus;
};

```

As you can gather from looking at the class definition, we still needed to use a templated member function pointer argument for the `AddCallback()` method. The use of `std::function`, however, isolates this to a single method, meaning we don't have to template the entire class, and that is an improvement. After the pointers to the method and the instance of the class, as well as a single placeholder that will be replaced by an argument in the future, are bound to a temporary function, we insert it into the callback container. Because of the way the compiler deals with the templated classes, we need to implement our template `AddCallback()` method in the header file, instead of the `.cpp` file. Just for the sake of consistency, and because it's a really simple method, we define `RemoveCallback()` in the header file too.

The other thing worthy of pointing out about the header is the implementation of the method that will be used to obtain the position of the mouse: `GetMousePos()`. It takes a pointer to a type of `sf::RenderWindow`, in case we want the coordinates returned to be relative to a specific window. The same window can also have or lose focus, so a flag `m_hasFocus` is kept around to keep track of that.

Implementing the event manager

Let's get started with actually implementing all the event manager class methods, starting, as always, with the constructor and destructor:

```
EventManager::EventManager() : m_hasFocus(true) { LoadBindings(); }
EventManager::~EventManager() {
    for (auto &itr : m_bindings) {
        delete itr.second;
        itr.second = nullptr;
    }
}
```

The constructor's job in this case is really simple. All it has to do is call a private method `LoadBindings()`, which is used to load the information about our bindings from a file. We will cover that shortly.

The destructor's job is also fairly run-of-the-mill for this type of class. If you recall, we store the bindings on the heap, so this dynamic memory has to be de-allocated.

Let's take a gander at the `AddBinding` method implementation:

```
bool EventManager::AddBinding(Binding *l_binding) {
    if (m_bindings.find(l_binding->m_name) != m_bindings.end())
        return false;

    return m_bindings.emplace(l_binding->m_name,
                              l_binding).second;
}
```

As you can see, it takes in a pointer to a binding. It then checks if the binding container already has a binding with the same name. If it does, the method returns `false`, which is useful for error-checking. If there are no name clashes, the new binding gets inserted into the container.

We have a way to add the bindings, but what about removing them? That's where the `RemoveBinding` method comes in:

```
bool EventManager::RemoveBinding(std::string l_name) {
    auto itr = m_bindings.find(l_name);
    if (itr == m_bindings.end()) { return false; }
    delete itr->second;
    m_bindings.erase(itr);
    return true;
}
```

It takes in a string argument and searches the container for a match to store into an iterator. If a match is found, it first frees up the memory by deleting the second element in the key-value pair, which is the dynamic memory allocated for the binding object, and then erases the entry from the container shortly before returning `true` for success. Easy.

As mentioned in the specifications for designing this class, we need a way to process the SFML events that are being polled in each iteration in order to look at them and see if there's anything in there we're interested in. This is where `HandleEvent` comes in:

```
void EventManager::HandleEvent(sf::Event& l_event) {
    // Handling SFML events.
    for (auto &b_itr : m_bindings) {
        Binding* bind = b_itr.second;
        for (auto &e_itr : bind->m_events) {
            EventType sfmlEvent = (EventType)l_event.type;
            if (e_itr.first != sfmlEvent) { continue; }
            if (sfmlEvent == EventType::KeyDown ||
                sfmlEvent == EventType::KeyUp)
            {
                if (e_itr.second.m_code == l_event.key.code) {
                    // Matching event/keystroke.
                    // Increase count.
                    if (bind->m_details.m_keyCode != -1) {
                        bind->m_details.m_keyCode = e_itr.second.m_code;
                    }
                    ++(bind->c);
                    break;
                }
            }
            else if (sfmlEvent == EventType::MButtonDown ||
                    sfmlEvent == EventType::MButtonUp)
            {

```

```
        if (e_itr.second.m_code == l_event.mouseButton.button) {
            // Matching event/keystroke.
            // Increase count.
            bind->m_details.m_mouse.x = l_event.mouseButton.x;
            bind->m_details.m_mouse.y = l_event.mouseButton.y;
            if (bind->m_details.m_keyCode != -1) {
                bind->m_details.m_keyCode = e_itr.second.m_code;
            }
            ++(bind->c);
            break;
        }
    } else {
        // No need for additional checking.
        if (sfmlEvent == EventType::MouseWheel) {
            bind->m_details.m_mouseWheelDelta =
                l_event.mouseWheel.delta;
        } else if (sfmlEvent == EventType::WindowResized) {
            bind->m_details.m_size.x = l_event.size.width;
            bind->m_details.m_size.y = l_event.size.height;
        } else if (sfmlEvent == EventType::TextEntered) {
            bind->m_details.m_textEntered = l_event.text.unicode;
        }
        ++(bind->c);
    }
}
}
```

It takes in, appropriately enough, an argument of type `sf::Event`. This method then has to iterate over all the bindings and through each event inside the binding to check if the type of the `l_event` argument matches the type of the binding event that's currently being processed. If it does, we check if it's a keyboard event or a mouse event, because that involves further checking for the keyboard keys or the mouse buttons matching our desired bindings. If it is either one of them, the last step is to check if either the keyboard key code or the mouse button code, which are respectively stored in the `l_event.key` and `l_event.mouseButton` structs, match the code of our binding event. With that being the case, or if it's a different type of event that doesn't require further processing, as demonstrated a few lines down, we increment the member `c` of the binding instance to signify a match shortly after relevant event information is stored in the event details structure of the binding.

Lastly, for input processing, we need to have an update method, which can handle real-time input checking as well as the validating and resetting of the states of the bindings. Let's write it:

```
void EventManager::Update() {
    if (!m_hasFocus) { return; }
    for (auto &b_itr : m_bindings) {
        Binding* bind = b_itr.second;
        for (auto &e_itr : bind->m_events) {
            switch (e_itr.first) {
                case (EventType::Keyboard) :
                    if (sf::Keyboard::isKeyPressed(
                        sf::Keyboard::Key(e_itr.second.m_code)))
                    {
                        if (bind->m_details.m_keyCode != -1) {
                            bind->m_details.m_keyCode = e_itr.second.m_code;
                        }
                        ++(bind->c);
                    }
                    break;
                case (EventType::Mouse) :
                    if (sf::Mouse::isButtonPressed(
                        sf::Mouse::Button(e_itr.second.m_code)))
                    {
                        if (bind->m_details.m_keyCode != -1) {
                            bind->m_details.m_keyCode = e_itr.second.m_code;
                        }
                        ++(bind->c);
                    }
                    break;
                case (EventType::Joystick) :
                    // Up for expansion.
                    break;
            }
        }

        if (bind->m_events.size() == bind->c) {
            auto callItr = m_callbacks.find(bind->m_name);
            if (callItr != m_callbacks.end()) {
                callItr->second(&bind->m_details);
            }
        }
        bind->c = 0;
        bind->m_details.Clear();
    }
}
```

Once again, we iterate over all the bindings and their events. In this case, however, we're only interested in `Keyboard`, `Mouse`, and `Joystick`, as those are the only devices we can check the real-time input of. Much like before, we check for the type of event we're dealing with, and use the appropriate class to check for the input. Incrementing the `c` member of the binding class, as usual, is our way of registering a match.

The final step is checking if the number of events in the event container matches the number of events that are "on". If that's the case, we locate our callback in the `m_callbacks` container and invoke the `second` data member with the parenthesis operator, because it is an `std::function` method wrapper, in turn officially implementing the callbacks. To it, we pass the address of the `EventDetails` structure that contains all the event information. Afterwards, it's important to reset the active event counter `c` to 0 for the next iteration because the state of any of the events checked previously could've changed and they all need to be re-evaluated.

Lastly, if you looked at the code top to bottom, you probably noticed that the case for controller input isn't doing anything. As a matter of fact, we don't even handle any events related to the controller. This is something that can be expanded later on and isn't vital to any of our projects. If you are eager to add support for joysticks and have access to one, consider it to be homework after this chapter.

Now that we have all this functionality, why not actually read in some binding information from a file? Let's take a look at the example configuration, named `keys.cfg`, that we will be loading in:

```
Window_close 0:0
Fullscreen_toggle 5:89
Move 9:0 24:38
```

This can be formatted in any way you want, however, for the sake of simplicity, the layout for it will remain pretty basic here. Each line is a new binding. It starts with the binding name, which is followed by the numerical representation of the event type enumeration and the code for the event separated by a colon. Every different event key:value pair is separated by spaces, as well as the binding name and the beginning of the events. Let's read this in:

```
void EventManager::LoadBindings() {
    std::string delimiter = ":";

    std::ifstream bindings;
    bindings.open("keys.cfg");
    if (!bindings.is_open()) {
        std::cout << "!! Failed loading keys.cfg." << std::endl;
        return;
    }
}
```

```

    }
    std::string line;
    while (std::getline(bindings, line)){
        std::stringstream keystream(line);
        std::string callbackName;
        keystream >> callbackName;
        Binding* bind = new Binding(callbackName);
        while (!keystream.eof()){
            std::string keyval;
            keystream >> keyval;
            int start = 0;
            int end = keyval.find(delimiter);
            if (end == std::string::npos){
                delete bind;
                bind = nullptr;
                break;
            }
            EventType type = EventType(
                stoi(keyval.substr(start, end - start)));
            int code = stoi(keyval.substr(end + delimiter.length(),
                keyval.find(delimiter, end + delimiter.length())));
            EventInfo eventInfo;
            eventInfo.m_code = code;

            bind->BindEvent(type, eventInfo);
        }

        if (!AddBinding(bind)){ delete bind; }
        bind = nullptr;
    }
    bindings.close();
}

```

We start by attempting to open the `keys.cfg` file. If it fails, this method spits out a console message notifying us about it. Next, we proceed into a `while` loop in order to read every single line in the file. We define an `std::stringstream` object, which allows us to nicely "stream" our string piece by piece, using the `>>` operator. It uses the default delimiter of a space, which is why we made that decision for the configuration file. After obtaining the name of our binding, we create a new `Binding` instance and pass the name in the constructor. Afterwards, by proceeding into a `while` loop and using `!keystream.eof()` as an argument, we make sure that it loops until the `std::stringstream` object reaches the end of the line it was reading. This loop runs once for each key:value pair, once again thanks to `std::stringstream` and its overloaded `>>` operator using whitespaces as delimiters by default.

After streaming in the type and code of an event, we have to make sure that we convert it from a string into two integer values, which are then stored in their respective local variables. It takes in parts of the string that got read in earlier in order to separate the key:value pair by splitting it at the delimiter character, which in this case was defined at the very top of this method as ":". If that character is not found within the string, the binding instance gets deleted and the line gets skipped, because it is most likely not formatted properly. If that's not the case, then the event gets successfully bound and the code moves on to the next pair.

Once all the values are read in and the end of the line is reached, we attempt to add the binding to the event manager. It is done in the if-statement in order to catch the error we talked about earlier relating to binding name clashes. If there is a clash, the binding instance gets deleted.

As you probably already know, it's also important to close the file after using it, so that's the last thing we do before this method concludes. With that done, our event manager is finally complete and it's time to actually put it to work.

Integrating the Event Manager class

Because the event manager needs to check all the events that get processed, it makes sense to keep it in our `Window` class, where we actually do the event polling. After all, the events that we're processing all originate from the window that's open, so it only makes sense to keep an instance of the event manager here. Let's make a slight adjustment to the `Window` class by adding a data member to it:

```
class Window{
public:
    ...
    bool IsFocused();
    EventManager* GetEventManager();
    void ToggleFullscreen(EventDetails* l_details);
    void Close(EventDetails* l_details = nullptr);
    ...
private:
    ...
    EventManager m_eventManager;
    bool m_isFocused;
};
```

In addition to adding an extra method for obtaining the event manager, the full screen toggle method has been modified to take in the `EventDetails` structure as an argument. A `Close` method is also added to our `Window` class, as well as a flag to keep track of whether the window is in focus or not. The method for closing the window itself is as simple as setting a single flag to `true`:

```
void Window::Close() { m_isDone = true; }
```

Now it's time to adjust the `Window::Update` method and pass in all the events being polled to the event manager:

```
void Window::Update() {
    sf::Event event;
    while(m_window.pollEvent(event)) {
        if (event.type == sf::Event::LostFocus) {
            m_isFocused = false;
            m_eventManager.SetFocus(false);
        }
        else if (event.type == sf::Event::GainedFocus) {
            m_isFocused = true;
            m_eventManager.SetFocus(true);
        }
        m_eventManager.HandleEvent(event);
    }
    m_eventManager.Update();
}
```

This ensures that every single event that ever gets dispatched in the window will be properly handled. It also notifies the event manager if the focus of the window changes.

Time to actually use the event manager! Let's do that in `Window::Setup` by registering two callbacks to some member functions, right after creating a new instance of the event manager:

```
void Window::Setup(...) {
    ...
    m_isFocused = true; // Default value for focused flag.
    m_eventManager->AddCallback("Fullscreen_toggle",
        &Window::ToggleFullscreen, this);
    m_eventManager->AddCallback("Window_close",
        &Window::Close, this);
    ...
}
```

Let's refer back to the `keys.cfg` file. We define the `Fullscreen_toggle` action and set up a key:value pair of 5:89, which essentially gets broken down to the event type of `KeyDown` (the number 5) and the code for the `F5` key on the keyboard (number 89). Both of these values are integer representations of the enumerations that we used.

The other callback that gets set up is for the action `Window_close`, which in the configuration file is bound to 0:0. The event type 0 corresponds to `Closed` in the enumeration table, and the code is irrelevant, so we just set that to 0 as well.

Both these actions get bound to methods of the `Window` class. Note the last argument in the `AddCallback` method, which is a `this` pointer referring to the current instance of the window. After successful compilation and launch, you should discover that hitting the `F5` key on your keyboard toggles the full screen mode of the window and clicking on the close button actually closes it. It works! Let's do something a little bit more fun with this now.

Moving a sprite revisited

Now that we have a fancy event manager, let's test it fully by moving a sprite to the location of the mouse when the left shift key is held down and the left mouse button is pressed. Add two new data members to your `Game` class: `m_texture` and `m_sprite`. Set them up as discussed in the previous chapters. For our purposes, we'll just be re-using the mushroom graphic from the first few chapters. Now add and implement a new method in your game class called `MoveSprite`:

```
void Game::MoveSprite(EventDetails* l_details) {
    sf::Vector2i mousepos =
        m_window->GetEventManager()->GetMousePos(
            m_window->GetRenderWindow());
    m_sprite.setPosition(mousepos.x, mousepos.y);
    std::cout << "Moving sprite to: "
        << mousepos.x << " "
        << mousepos.y << std::endl;
}
```

What we do here is grab the mouse position relative to the current window from the event manager and store it in a local integer vector called `mousepos`. We then set the position of our sprite to the current mouse position and print out a little sentence in the console window. Very basic, but it will serve nicely as a test. Let's set up our callback:

```
Game::Game() {
    ...
    // Texture and sprite setup.
```

```

...
m_window->GetEventManager()->AddCallback("Move",
    &Game::MoveSprite,this);
}

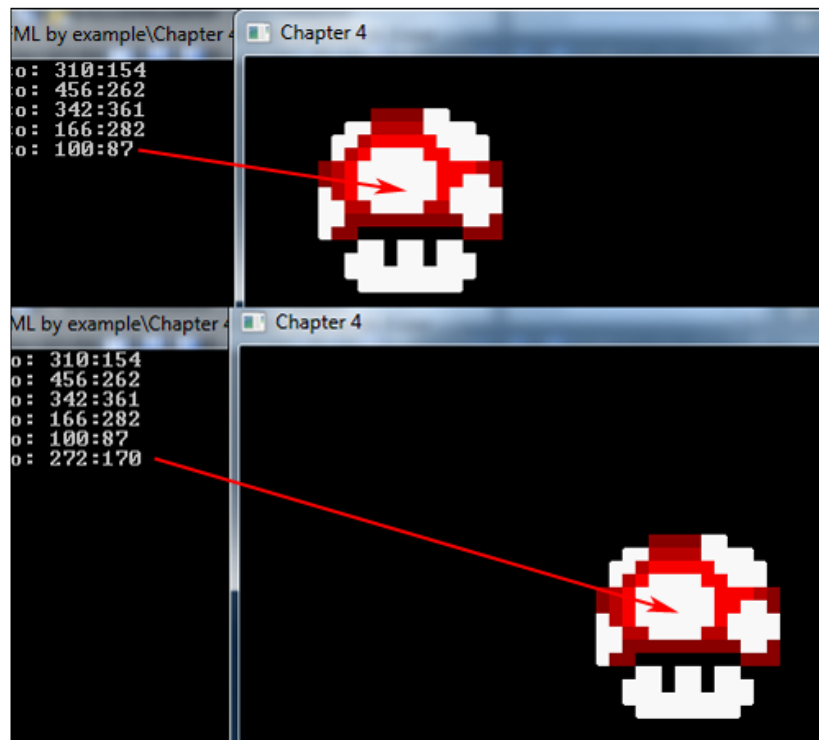
```

We tie the action name `Move` to the `MoveSprite` method of the `Game` class and pass in a pointer to the current instance, just like before. Before running this, let's take a peek at the way the move action is defined in the `keys.cfg` file:

```
Move 9:0 24:38
```

The first event type corresponds to `MButtonDown`, which is the event of the left mouse button being pressed down. The second event type corresponds to the `Keyboard` event, which checks for real-time input through the `sf::Keyboard` class. The number 38 is the left shift key code, corresponding to `sf::Keyboard::LShift`.

Upon compilation and execution of our application, we should end up with a sprite being rendered on the screen. If we hold the left shift key and left click anywhere on the screen, it will magically move to that position!



Principles of use

Knowing when to use which types of events is important even in this design. Let's say, for example, that you only want a callback to be called once for a binding that involves the left shift and the *R* key. You wouldn't define both the event types as `Keyboard`, because that would keep invoking the callback method as long as these keys are down. You also don't want to define both of them as `KeyDown` events, because that would mean that both of these events would have to be registered at the same time, which, when holding down multiple keys, is likely not going to happen because of the screen refresh rate. The correct way to use this is mixing the `Keyboard` and `KeyDown` events so that the very last key to be pressed is the `KeyDown` type and the rest of the keys will be `Keyboard` types. In our example, it means that we would have the left shift key being checked through the `sf::Keyboard` class, while the *R* key would default to an event being dispatched. That might sound odd at first, however, consider the famous example of the key combination *Ctrl + Alt + Del* on your computer. It works that way, but if you hold the keys in reverse order, it would do nothing. If we were implementing this functionality, we'd most likely make sure that the *Ctrl* and *Alt* keys are always checked through the `sf::Keyboard` class, while the *Del* key is registered through the event polling.

One last thing to note as far as the use of this class goes, is that some events aren't yet supported, such as the `sf::Event::TextEntered` event because additional information is required in order to fully utilize them, which is obtained from the `sf::Event` class. Proper expansion of the event manager to support these features will be covered in the later chapters, once we're dealing with problems that require the said events.

Common mistakes

One of the most common mistakes the newcomers make when it comes to SFML input is using certain methods of checking the user input for the wrong tasks, such as using the window events for real time character movement or capturing text input. Understanding the limitations of anything you use is the key to cultivating any kind of decent performance. Make sure to stick to the intended uses of all the different mechanisms we've discussed in order to achieve optimal results.

Another fairly common mistake people make is defining templates in the `.cpp` file instead of the header. If you are getting linking errors pertaining to a method that just so happens to utilize templates, such as the `EventManager::AddCallback()` method, make sure to move the implementation of the method and the definition of the template right to the header of your class, otherwise the compiler cannot instantiate the template and the method becomes inaccessible during the linking process.

Lastly, a rather simple yet extremely popular mistake lots of new users of SFML are guilty of is not knowing how to correctly obtain the mouse coordinates that are relative to the window. It ranges from simply using the wrong coordinates and experiencing weird behavior to grabbing the coordinates relative to the desktop as well as the position of the window and subtracting one from another to obtain the local mouse position. While the latter works, it's a bit excessive, especially since SFML already provides you with a way to do it without reinventing the wheel. Simply pass in a reference of your window to the `sf::Mouse::getPosition()` method. That's all you need.

Summary

Much like proper code organization, robust input management is one of the many things that can mean the difference between you happily developing an application and the same application drowning in the sea of other failed projects. With proper and flexible design comes great code reusability, so congratulations on taking yet another step towards building an application that will not bite the dust simply because it was painful to work with due to its myopic construction.

There is no design in this world that's inarguably perfect, however, with this chapter coming to fruition we are now yet another step closer to the goal that we set for ourselves at the very beginning of this experience. This goal varies between individuals. Maybe it has grown since we started; it may have even manifested itself into something completely different than it was before. None of that is certain to the rest of us, but it doesn't really matter. What matters is that we are in full control of where we take those goals, even if we have no control of where they take us. And while this journey towards our goals continues, and even as the new ones begin to emerge, we can now say that we have stronger means of taking control over the entire process, much like we built our own stronger means of taking control of our applications. So, move yourself forward to the next chapter and resume your journey, by learning about application states. We'll see you there!

